



Section 1 / 9

Introduction to GraphQL

GraphQL is a query language typically used by web APIs as an alternative to REST. It enables the client to fetch required data through a simple syntax while providing a wide variety of features typically provided by query languages, such as SQL. Like REST APIs, GraphQL APIs can read, update, create, or delete data. However, GraphQL APIs are typically implemented on a single endpoint that handles all queries. As such, one of the primary benefits of using GraphQL over traditional REST APIs is the efficiency in resource utilization and request handling.

Basic Overview

A GraphQL service typically runs on a single endpoint to receive queries. Most commonly, the endpoint is located at `/graphql`, `/api/graphql`, or a similar URL. For frontend web applications to use this GraphQL endpoint, it needs to be exposed. Just like REST APIs, we can, however, interact with the GraphQL endpoint directly without going through the frontend web application to identify security vulnerabilities.

From an abstract point of view, GraphQL queries select **fields** of objects. Each object is of a specific **type** defined by the backend. The query is structured according to GraphQL syntax, with the name of the **query** to run at the root. For instance, we can query the **id**, **username**, and **role** fields of all **User** objects by running the **users** query:

```
{  
  users {
```

graphql

```
    id
    username
    role
  }
}
```

The resulting GraphQL response is structured in the same way and might look something like this:

```
{
  "data": {
    "users": [
      {
        "id": 1,
        "username": "htb-stdnt",
        "role": "user"
      },
      {
        "id": 2,
        "username": "admin",
        "role": "admin"
      }
    ]
  }
}
```

graphql

If a query supports arguments, we can add a supported argument to filter the query results. For instance, if the query `users` supports the `username` argument, we can query a specific user by supplying their username:

```
{
  users(username: "admin") {
    id
    username
    role
  }
}
```

graphql



```
}
```

We can add or remove fields from the query we are interested in. For instance, if we are not interested in the `role` field and instead want to obtain the user's password, we can adjust the query accordingly:

graphql

```
{
  users(username: "admin") {
    id
    username
    password
  }
}
```

Furthermore, GraphQL queries support sub-querying, which enables a query to retrieve details from an object that references another object. For instance, assume that a `posts` query returns a field `author` that holds a user object. We can then query the username and role of the `author` in our query like so:

graphql

```
{
  posts {
    title
    author {
      username
      role
    }
  }
}
```

The result contains the `title` of all posts as well as the queried data of the corresponding author:

graphql

```
{
  "data": {
    "posts": [
      {
        "title": "Hello World!",
        "author": {
          "username": "htb-stdnt",
          "role": "user"
        }
      },
      {
        "title": "Test",
        "author": {
          "username": "test",
          "role": "user"
        }
      }
    ]
  }
}
```

GraphQL queries support much more complex operations. However, this introductory overview is sufficient for the purposes of this module. For more details, check out the [Learn](#) section on the official GraphQL website.

1 / 9 Sections



Mark Complete & Next